LUT
University

CT60A5401 GAME DEVELOPMENT PROJECT

ACADEMIC YEAR 2022-2023

Assoc. Prof. Jussi Kasurinen (Adj. prof. of entertainment software engineering)

TODAY'S AGENDA

GAME ENGINES

RE: WHAT DO YOU NEED TO MAKE GAMES?

In short: Computer, developer and a place to sit

In more detail:

- Game designers
- Graphics, animation artists
- Programmers
- Test audiences
- Programming tools (version control, code editors)
- **Game engine or other framework**
- Auxiliary services such as physics libraries, payment management systems, cloud services etc.

RE: TOOLS IN DIFFERENT STAGES OF DEVELOPMENT

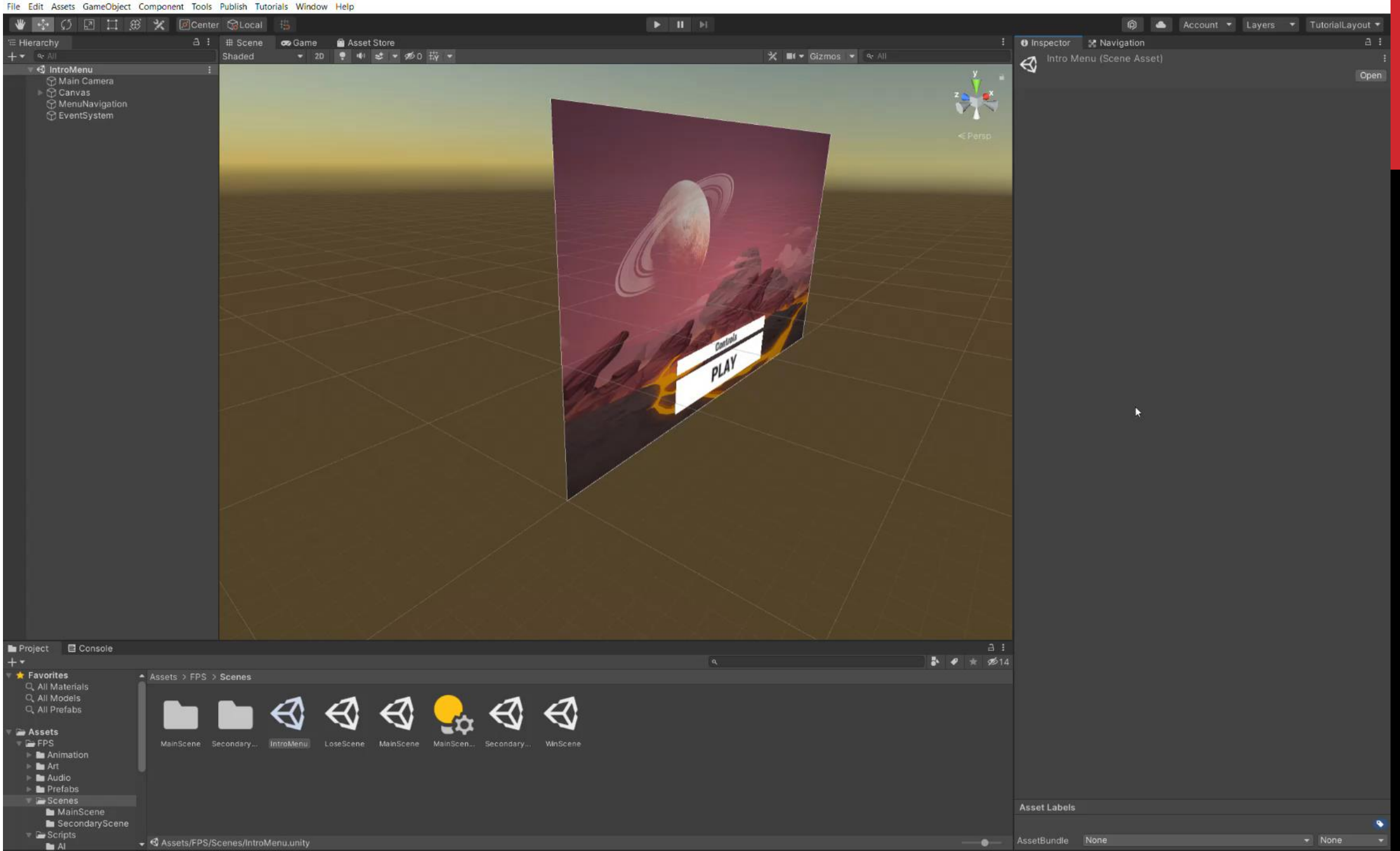
Activity	Preproduction, design phase	Production, implementation phase
Design, prototyping tools	Documentation tools, Doc. distribution, graphics software, development tools, **game engine**	Development tools, **game engine**, graphics software
Implementation, development tools	Graphics software, development tools, **game engine**, cloud services	**Game engine**, development tools, graphics software, audio software
Project support tools	Documentation, cloud services, file sharing service	Version control, cloud services, file sharing service, ticket systems

ON GAME ENGINES

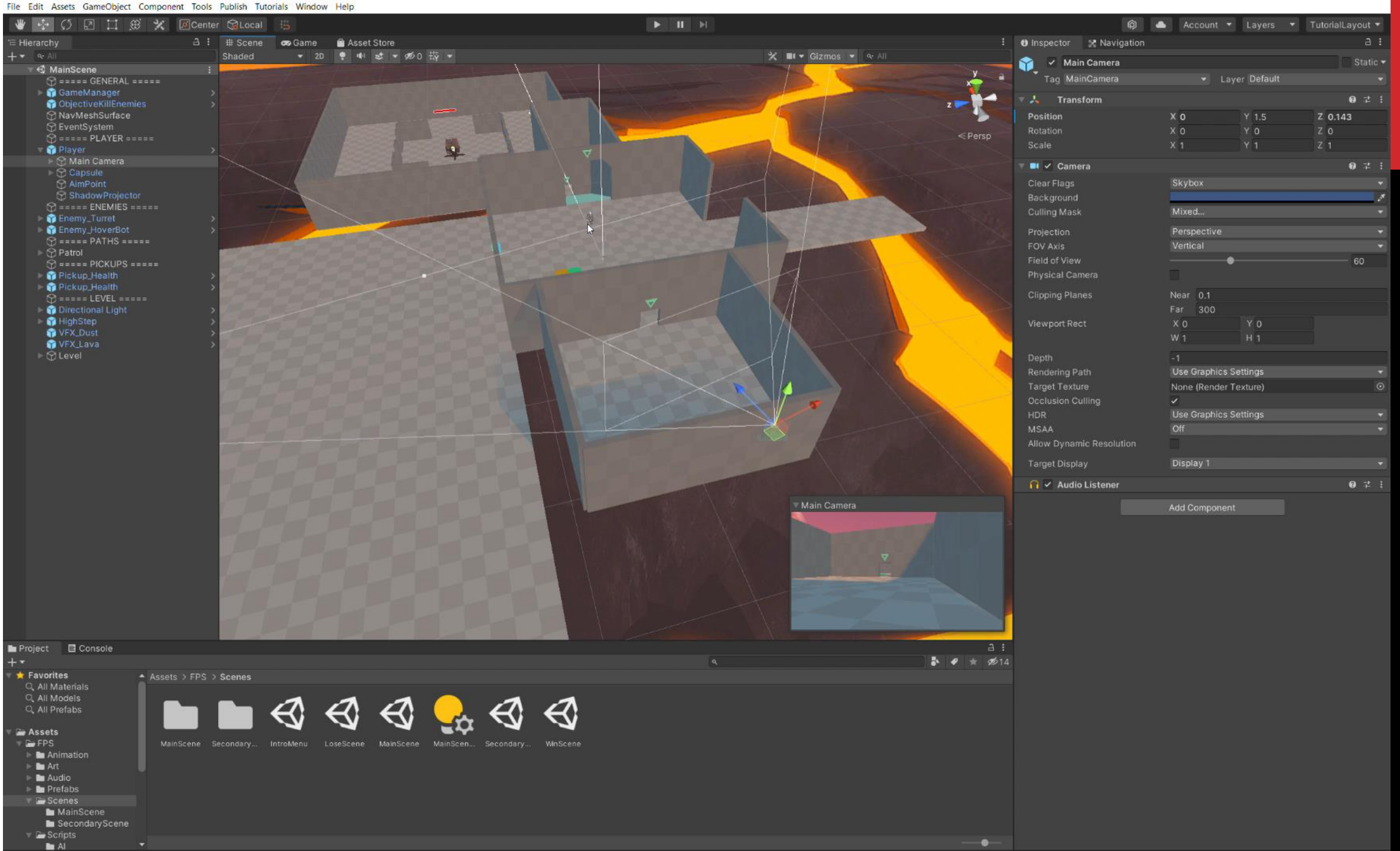
- **It is safe to say that Game Engines are the most crucial pieces of software the game studio has.**
- **However, that does not mean that the “Game Engines” are all the same:**
  - Scope (Aux library to Full development suite)
  - Platform (Not all engines support all game delivery platforms)
  - Genre (Some Game Engines are specialized to certain types of genres)
  - Licensing (commercial or open source, service or product)
  - Accessibility (programming platform or “constructor set”)

EXAMPLES ON GAME ENGINES

All examples from freely available
Game Engine tutorials, or self-made

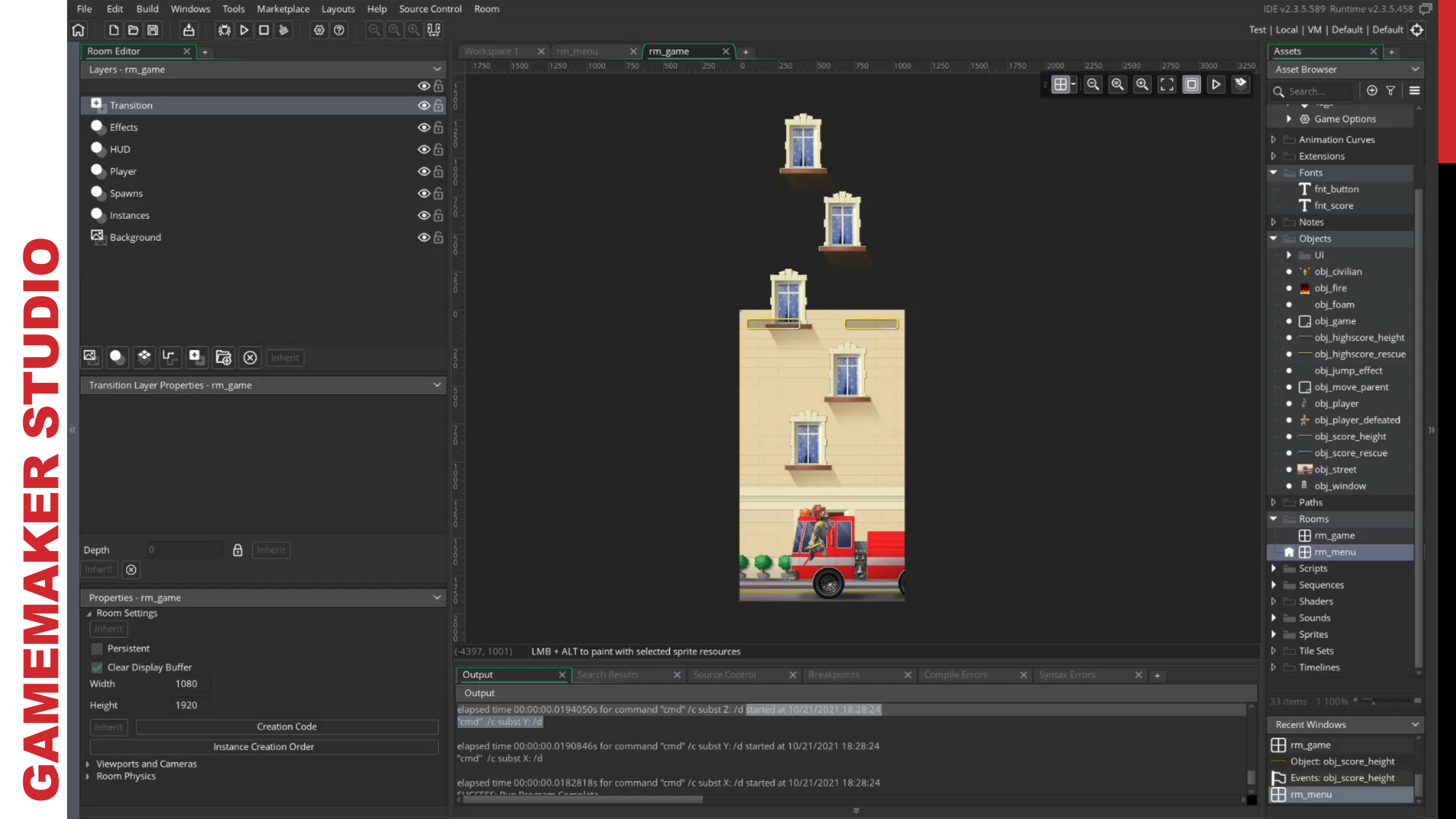

File Edit Assets GameObject Component Tools Publish Tutorials Window Help
Center Local
Account Layers TutorialLayout
Hierarchy
IntroMenu
Main Camera
Canvas
MenuNavigation
EventSystem
Scene Game Asset Store
Shaded 2D Gizmos
Persp
Controls
PLAY
Inspector Navigation
Intro Menu (Scene Asset)
Open
Project Console
Favorites
All Materials
All Models
All Prefabs
Assets
FPS
Animation
Art
Audio
Prefabs
Scenes
MainScene
SecondaryScene
Scripts
AI
Assets > FPS > Scenes
MainScene
Secondary...
IntroMenu
LoseScene
MainScene
MainScen...
Secondary...
WinScene
Assets/FPS/Scenes/IntroMenu.unity
Asset Labels
AssetBundle
None
None

UNITY


File Edit Assets GameObject Component Tools Publish Tutorials Window Help
Hierarchy
MainScene
GameManager
ObjectiveKillEnemies
NavMeshSurface
EventSystem
Player
Main Camera
Capsule
AimPoint
ShadowProjector
Enemy_Turret
Enemy_HoverBot
Patrol
Pickup_Health
Pickup_Health
Directional Light
HighStep
VFX_Dust
VFX_Lava
Level
Scene
Game
Asset Store
Persp
Main Camera
Inspector
Navigation
Main Camera
Transform
Position
Rotation
Scale
Camera
Clear Flags
Skybox
Background
Culling Mask
Projection
Perspective
FOV Axis
Vertical
Field of View
Physical Camera
Clipping Planes
Viewport Rect
Depth
Rendering Path
Target Texture
Occlusion Culling
HDR
MSAA
Allow Dynamic Resolution
Target Display
Audio Listener
Add Component
Project
Console
Favorites
Assets
FPS
Animation
Art
Audio
Prefabs
Scenes
MainScene
SecondaryScene
Scripts
AI
Assets > FPS > Scenes
MainScene
Secondary...
IntroMenu
LoseScene
MainScene
MainScen...
Secondary...
WinScene

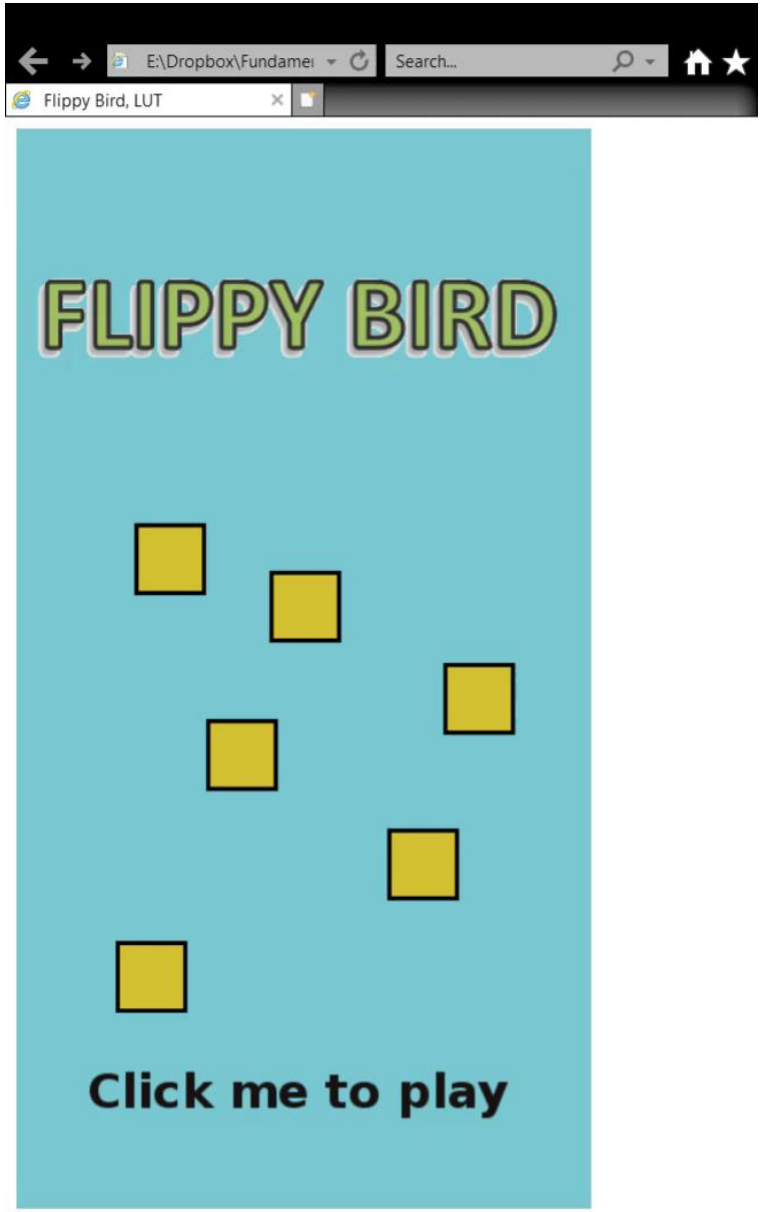
UNITY

GAMEMAKER STUDIO



GAMEMAKER STUDIO

FLIPPY BIRD



One of the old example games from the course “Game Development Project”
Rest of them are available at www2.it.lut.fi/GRIP

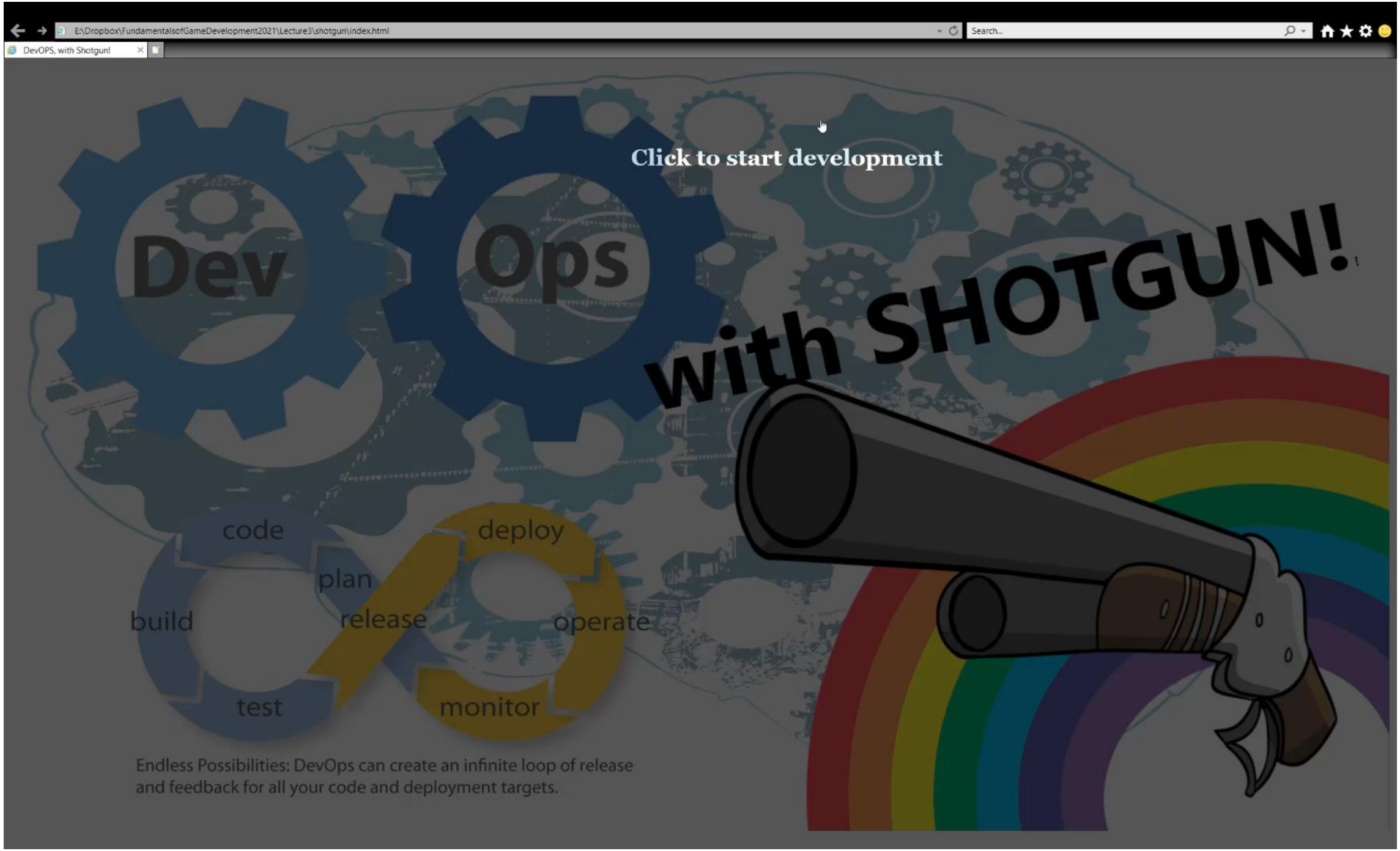
PHAZER AND BOX2D

File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?

main.js | phaser.min.js | box2d.js

```
// Initialize Phaser, and create a 400x490px game
var game = new Phaser.Game(400, 490, Phaser.AUTO, 'gameDiv');

// Create our 'main' state that will contain the game
var mainState = {

    preload: function() {
        // This function will be executed at the beginning
        // That's where we load the game's assets
        game.stage.backgroundColor = '#71c5cf';
        game.load.image('bird', 'assets/bird.png');
        game.load.image('pipe', 'assets/pipe.png');
    },

    create: function() {
        // This function is called after the preload function
        // Here we set up the game, display sprites, etc.
        game.physics.startSystem(Phaser.Physics.ARCADE);

        //Display the bird
        this.bird = this.game.add.sprite(100,245,'bird');

        game.physics.arcade.enable(this.bird);
        this.bird.body.gravity.y = 1000;
        dead = false;
        spaceKey = game.input.keyboard.addKey(Phaser.Keyboard.SPACEBAR);
        spaceKey.onDown.add(this.jump, this)
        this.pipes = game.add.group(); // Create a group
        this.pipes.enableBody = true;  // Add physics to the group
        this.pipes.createMultiple(20, 'pipe'); // Create 20 pipes
        this.timer = game.time.events.loop(1600, this.addRowOfPipes, this);

        this.score = 0;
        this.labelScore = game.add.text(20, 20, "0", { font: "30px Arial", fill: "#ffffff" });
    },

    update: function() {
        // This function is called 60 times per second
        // It contains the game's logic
        if (this.bird.inWorld == false)
            this.waitresetGame();
        game.physics.arcade.overlap(this.bird, this.pipes, this.waitresetGame, null, this);
        if (dead == true && spaceKey.isDown){
            dead = false;
            this.restartGame();
```

NO ENGINE


E:\Dropbox\FundamentalsofGameDevelopment2021\Lecture3\shotgun\index.html
Search...
DevOPS, with Shotgun!
Click to start development
Dev
Ops
with SHOTGUN!
code
plan
build
release
test
deploy
operate
monitor
Endless Possibilities: DevOps can create an infinite loop of release and feedback for all your code and deployment targets.

SUMMARY

- **It is safe to say that Game Engines are the most crucial pieces of software the game studio has.**
- **On this course, the idea is not to just learn to use one game engine, but to understand what components make a game:**
  - Levels/Scenes
  - Animations
  - Objects
  - Script
  - SFX…
- **And these are usually, but not always, combined into a product by using an engine.**
- **However, it would be prudent to pick one, and learn that well but we will get back to that on our next lecture.**

COURSE SCHEDULE

So what you should have now that the course is at midpoint schedule-wise?

- **Project 1 done, game engine for own game selected.**
- **Own Game idea going forward, game design document draft created.**
- **Remember the weekly advice sessions, and possibility to ask help from the course Moodle pages.**

CT60A5401 GAME DEVELOPMENT PROJECT

Comments, feedback, questions?

Email jussi.kasurinen@lut.fi